DomainSleuth: Misconfiguration Hunting

By Haydn Kuti

Project Summary: SleuthHound

The primary goal of this module is to automate the identification of critical misconfigurations within Active Directory. By parsing SharpHound JSON data through a custom logic engine, it transforms raw data into an actionable audit report. This streamlines the work of security professionals, and allows the detection of high-risk vulnerabilities before they're exploited.
Core detection capabilities:

- **Excessive User Rights**: Identifies unprivileged accounts with dangerous permissions, such as GPO modification or object ownership.
- **Unconstrained Delegation**: Detects services that can impersonate users, creating a high risk for administrative credential theft.
- **AS-REP Roasting**: Flags accounts that don't require Kerberos pre-authentication, making them vulnerable to offline password cracking.
- **Machine Account Quota**: Locates domains allowing any authenticated user to join multiple rogue machines to the network.
- **Outdated OS Versions**: Highlights domain Controllers running unpatched or end-of-life operation systems vulnerable to known exploits.

Vulnerability One: Excessive User Rights & Permissions: Technical Logic & Indicators

Primary ACE Flags (High-Impact Control)

- **GenericAll**: Grants full object control, including password resets and group/computer takeover.
- **GenericWrite**: Permits updates to non-protected attributes; commonly abused for Kerberoasting.
- **WriteDacl**: Allows modification of security permissions, enabling users to grant themselves full control.
- **WriteOwner**: Enables taking ownership of an object to subsequently overwrite its security settings.
- **AllExtendedRights**: Covers sensitive tasks like DCSync attacks and administrative password resets.

Local & Lateral Movement Rights

- **AdminTo**: Identifies accounts with local administrative privileges on remote systems.
- **CanRDP**: Flags users permitted to access systems via Remote Desktop Protocol.
- **SeRemoteInteractiveLogon Rights**: The underlying User Rights Assignment (URA) that enables RDP access.
- **CanPSRemote**: Pinpoints users authorized to execute remote commands via PowerShell.

Hidden Control Relationships

- **AddMember**: Found in Group Objects; allows adding users to a group to inherit its total permissions.
- **ForceChangePassword**: Explicitly allows resetting a target's password without knowledge of the original.

Vulnerability One: Excessive User Rights & Permissions: Security Risks

Privilege Escalation & Lateral Movement

- **Attack Path**: Compromised standard accounts with high-level ACEs (like `GenericAll`) allow for rapid escalation to Domain Admin.
- **Network Traversal**: Misconfigured group nesting and over-privileged service accounts provide threat actors with the keys they need to move through the network.

Increased Attack Surface

- **Target Density**: More privileges accounts create more high-value targets for credential theft and malware deployment.
- **Hidden Risks**: Inactive or service accounts with “Debug programs” rights allow threat actors to hijack system-level processes.

Insider Threats & Privilege Gaps

- **Abuse of Access**: Disgruntled or negligent employees can leverage excessive rights for data theft, fraud, or intentional sabotage.
- **Permission Accumulation**: Privilege creep occurs when rights aren’t revoked after role changes, leaving dangerous, undocumented access paths open.

Compliance & Operational Impact

- **Audit Violations**: Excessive, unreviewed permissions violate regulatory standards, leading to financial penalties.
- **Data Integrity**: Over-privileged accounts (often lacking MFA or frequent password rotation) are primary vectors for unauthorized data modification or deletion.

Vulnerability One: Excessive User Rights & Permissions: Mitigation & Operational Best Practices

1. Core Mitigation Strategies

- **Enforce Least Privilege**: Grant only the minimum permissions required for specific job functions to prevent privilege creep.
- **Protect High-Privilege Groups**: Strictly limit and audit membership in sensitive groups like Domain, Enterprise, and Schema Admins.
- **Tiered Administration**: Use hardened, separate accounts for administrative tasks; never use admin accounts for daily tasks, like email or browsing.
- **Delegation of Control**: Use the AD Delegation Wizard to assign specific, limited rights (such as password resets) to specific OUs rather than full administrative access.
- **Secure Service Accounts**: Implement Group Managed Service Accounts (gMSAs) to automate password rotation and reduce compromise risks.

2. Operational Best Practices

- **Account Lifecycle Management**: Regularly audit and remove inactive, disabled, or stale accounts (no login for 90+ days).
- **Restricted Groups**: Use GPO “Restricted Groups” to automatically remove unauthorized users from critical security groups.
- **LAPS Implementation**: Deploy Local Administrator Password Solution to randomize local admin passwords, effectively blocking lateral movement.
- **Active Monitoring**: Enable detailed AD DS auditing and monitor for sensitive rights like `DS-Replication-Get-Changes` to detect DCSync attacks.

Vulnerability One: Excessive User Rights & Permissions: The Logic Engine

1. **Identification Strategy**
   - **Target Data**: The function iterates through the users list extracted from SharpHound JSON files.
   - **Property Extraction**: It pulls the name property for reporting and looks specifically into the Aces (Access Control Entries) array.

2. Filtering Dangerous Rights
   - **The Watchlist**: The script defines a list of high-risk ACE flags: `GenericAll`, `GenericWrite`, `WriteDacl`, `WriteOwner`, and `AllExtendedRights`.
   - **Deduplication**: A Python `set()` is utilized to store `found_rights`. This ensures that even if a user has multiple instances of a specific right, it's only flagged once per user.

3. **Reporting and Output**
   - **Conditional Flagging**: A vulnerability entry is only created and appended to the flags list if the `found_rights` set is not empty.
   - **Structured Data**: The output includes the specific username, the vulnerability type, a list of the exact permissions found, and a human-readable description for the final report.

```python
def check_excessive_permissions(users, flags):

    dangerous_rights = [
        "GenericAll",
        "GenericWrite",
        "WriteDacl",
        "WriteOwner",
        "AllExtendedRights"
    ]

    for user in users:

        properties = user.get("Properties", {})
        username = properties.get("name", "UnknownUser")

        aces = user.get("Aces", [])

        # Use a SET to prevent duplicates
        found_rights = set()

        for ace in aces:

            right = ace.get("RightName")

            if right in dangerous_rights:
                found_rights.add(right)

        # Only add a flag if something dangerous was found
        if found_rights:

            flags.append({
                "user": username,
                "vulnerability": "Excessive Permissions",
                "permissions": list(found_rights),
                "description": f"{username} has dangerous
rights: {', '.join(found_rights)}"
            })
```

Vulnerability Two: Insecure Delegation: Technical Logic & Delegation Types

1. Unconstrained Delegation

The Mechanism: The server can impersonate any user who authenticates to it and forward those credentials anywhere in the network.

Search Criteria: Look for the `"unconstraineddelegation": true` Boolean within the `Properties` block of Computer or User objects.

Logic: Any non-Domain Controller with this flag is a high-value target for harvesting Ticket Granting Tickets (TGTs) from privileged users.

2. Constrained Delegation

The Mechanism: Restricts impersonation to specific services, but remains abusable if the service account is compromised.

Search Criteria: Identify the `AllowedToDelegate` Edge or the `msds-allowedtodelegateto` attribute.

Vulnerability: If an attacker gains `GenericWrite` or `GenericAll` over an account with these rights, they can escalate to the specific services listed in the SPN attribute.

3. Resource-Based Constrained Delegation

The Mechanism: The target resource (like a computer) defines which accounts are allowed to delegate to it.

Search Criteria: Identify the `AllowedToAct` Edge, which maps to the `msDS-AllowedToActOnBehalfOfOtherIdentity` attribute.

Vulnerability: Attackers with write permissions over a computer object can modify this attribute to point to their own account, effectively granting themselves administrative access to that machine.

Vulnerability Two: Insecure Delegation: Security Risks

1. The Path to Full Domain Compromise

- **Ticket Harvesting**: If a server with unconstrained delegation is compromised, attackers can extract cached Ticket-Granting Tickets (TGTs) from memory using tools like Mimikatz.
- **Impersonation**: Once a TGT is stolen, the attacker can impersonate that user, including Domain Admins, to any service in the forest.

2. Advanced Lateral Movement & Coercion

- **The Printer Bug**: Attackers can exploit legacy services to force a Domain Controller (DC) to authenticate to a compromised, unconstrained machine. This immediately hands the DC's credentials to the attacker.
- **Credential-less Traversal**: Because delegation allows the service to act as the user, attackers can move between systems without ever needing to crack or steal an actual password.

3. Persistence & Infrastructure Risk

- **Stealthy Persistence**: By impersonating high-privileged users, attackers can create backdoors and persistent access that appear as legitimate administrative activity.
- **Machine Account Exploitation**: This risk isn't limited to user accounts; compromised computer accounts with delegation can impersonate other sensitive machines that often hold high-level domain privileges.

Vulnerability Two: Insecure Delegation: Mitigation & Defense-in-Depth

1. Core Remediation Steps
Disable Unconstrained Delegation: Transition systems from "Trusted for delegation" to "Do not trust this computer for delegation" within ADUC.

Enforce Modern Delegation: Where delegation is required, replace legacy unconstrained methods with Resource-Based Constrained Delegation (RBCD) to strictly limit impersonation scope.

Account Hardening: Enable the attribute "Account is sensitive and cannot be delegated" for all high-privileged accounts to explicitly block impersonation.

Protected Users Group: Add administrative accounts to the Protected Users Security Group to enforce non-cacheable credentials and restricted delegation.

2. Specialized Threat Mitigation
Block Coercion Paths: Disable the Print Spooler service on Domain Controllers to prevent attackers from forcing the DC to authenticate to a compromised, unconstrained server.

Permission Auditing: Regularly monitor which users/groups hold the rights to enable delegation, as this privilege is often a silent path to domain compromise.

Delegation Type	Security Level	Primary Risk
Unconstrained	Legacy/Insecure	**Critical:** Threat actors can impersonate any user to any service.
Constrained	Moderate	**Medium:** Restricted to specific services but still abusable.
Resource-Based	Most Secure	**Low:** Access is denied at the target; hardest to exploit.

Vulnerability Two: Insecure Delegation: The Logic Engine

```
def check_unconstrained_delegation(users, flags):

    for user in users:

        properties = user.get("Properties", {})

        username = properties.get("name",
"UnknownUser")

        if properties.get("unconstraineddelegation") ==
True:

            flags.append({
                "user": username,
                "vulnerability": "Unconstrained Delegation",
                "description": f"{username} has unconstrained
delegation enabled (high risk)."
            })
```

1. Identification Strategy

- **Target Data:** This function scans the `users` (and can be adapted for `computers`) collection within the SharpHound dataset.
- **Property Focus:** It looks specifically at the `Properties` object, which contains Boolean flags for various Kerberos settings.

2. Boolean Logic Detection

- **The Flag:** The script checks for the specific key `"unconstraineddelegation"`.
- **Explicit Matching:** It verifies if the value is explicitly `True`. In Active Directory, this corresponds to the `TRUSTED_FOR_DELEGATION` flag in the User Account Control (UAC) attribute.

3. Reporting and Output

- **High-Risk Tagging:** Every account identified is labeled with a high risk warning in the description due to the potential for credential harvesting.
- **Automation Benefit:** This replaces the manual effort of hunting through thousands of lines of JSON, instantly isolating only the accounts that pose a delegation threat.

Vulnerability Three: AS-REP Roasting (Kerberos Pre-Auth): Technical Logic & Vulnerability Analysis

1. The Identification Property

JSON Indicator: The parser searches for the `"dontreqpreauth": true` key-value pair within the `Properties` block of user objects.

AD Mapping: This property maps directly to the `DONT_REQ_PREAUTH` flag within the `userAccountControl` attribute in Active Directory.

2. The Attack Mechanism

Credential-less Request: Attackers only need a valid username to request a Kerberos ticket; they do not need to be authenticated to the domain.

Encrypted Response: The Domain Controller sends an AS-REP response containing data encrypted with the user's password hash.

Offline Cracking: Threat actors capture this response and use tools like **Hashcat (Mode 18200)** or **John the Ripper** to crack the password offline, bypassing network-based account lockout policies and detection.

3. Common Root Causes

Legacy Compatibility: Often left enabled for older applications that do not support modern Kerberos pre-authentication.

Service Account Misconfiguration: Frequently found on service accounts where security settings were overlooked during setup.

Account Migration Oversights: Artifacts left over from migrating accounts between domains or forests.

Vulnerability Three: AS-REP Roasting (Kerberos Pre-Auth): Security Risk & Defense Strategy

1. Critical Security Risks

- **Zero-Credential Entry:** Attackers do not need an existing domain account to perform the initial request; only a valid username is required.
- **KDC Exploitation:** The Key Distribution Center (KDC) is forced to return a TGT encrypted with the user's password hash *before* identity verification occurs.
- **Offline Stealth:** Because the intensive cracking happens on the attacker's local machine, it bypasses network-based account lockout policies and SIEM alerts.
- **High Impact:** Frequently utilized by Advanced Persistent Threat (APT) groups for rapid lateral movement and initial credential theft.

2. Detection & Mitigation Strategies

- **Enforce Pre-Authentication:** This is the primary mitigation strategy. Audit and disable the "Do not require Kerberos preauthentication" setting for all user accounts via Active Directory Users and Computers (ADUC).
- **PowerShell Auditing:** Proactively identify vulnerable accounts using: `Get-ADUser -Filter 'DoesNotRequirePreAuth -eq $true'`
- **Event Monitoring:** Configure alerts for **Windows Security Event ID 4768** (Kerberos ticket requested), paying close attention to instances where pre-authentication is bypassed.
- **Password Complexity:** For any legacy accounts where this setting is a functional requirement, enforce extreme password complexity to make offline cracking mathematically infeasible.

Vulnerability Three: AS-REP Roasting (Kerberos Pre-Auth): The Logic Engine

1. **Identification Strategy**

- **Target Data:** The function processes the `users` array within the SharpHound JSON output.
- **Target Attribute:** It isolates the `Properties` block of each user object to check for authentication requirements.

2. Flag-Based Detection

- **The "Roastable" Key:** The logic specifically hunts for the `"dontreqpreauth"` key.
- **Boolean Evaluation:** If the value is explicitly `True`, the script identifies that the `DONT_REQ_PREAUTH` bit is flipped in the user's Account Control settings.

3. Reporting and Output

- **Vulnerability Mapping:** Impacted users are categorized under the "AS-REP Roasting" vulnerability type.
- **Actionable Description:** The generated report explicitly states that the user is "AS-REP roastable," providing immediate context for blue team remediation efforts.

```
def check_asrep_roasting(users, flags):

    for user in users:

        properties = user.get("Properties", {})

        username = properties.get("name",
"UnknownUser")

        # Check AS-REP flag
        if properties.get("dontreqpreauth") == True:

            flags.append({
                "user": username,
                "vulnerability": "AS-REP Roasting",
                "description": f"{username} does not
require Kerberos pre-authentication (AS-REP
roastable)."
            })
```

Vulnerability Four: Machine Account Quota: Technical Logic & Domain Analysis

1. The Identification Property

- **JSON Location:** The parser examines the `domains.json` file rather than individual user or computer files.
- **Key Indicator:** It hunts for the `"machineaccountquota"` key within the domain object's `Properties` block.
- **Logic Threshold:** While the default value is **10**, any value **greater than 0** is flagged as a vulnerability.

2. The Attack Surface

- **Object Creation:** When this quota is active, any authenticated user can join new computer objects to the domain without administrative approval.
- **SPN Generation:** Created machine accounts automatically receive a Service Principal Name (SPN), which is a critical prerequisite for advanced Kerberos attacks.
- **Exploitation Path:** This is most commonly leveraged as a first step for **Resource-Based Constrained Delegation (RBCD)** attacks, allowing an attacker to grant themselves administrative rights over a target system.

3. Key Attribute Mapping

- **LDAP Attribute:** In the live Active Directory environment, this property maps directly to `ms-DS-MachineAccountQuota`.
- **Visibility:** Identifying this at the domain level provides a global "health check" on the security posture regarding rogue device registration.

Vulnerability Four: Machine Account Quota: Security Risk Analysis

1. Unauthorized Network Foothold

- **Low-Barrier Entry:** Attackers with standard user credentials can spawn up to 10 computer objects without administrative approval.
- **Silent Presence:** These machine accounts are legitimate AD objects, making them a "silent" foothold that blends in with standard network traffic.

2. Advanced Exploitation Paths

- **RBCD Foundation:** Creating a rogue machine account is the primary prerequisite for **Resource-Based Constrained Delegation** attacks. Attackers use these accounts to configure delegation rights and impersonate high-privileged users.
- **Shadow Credentials:** Attackers can inject credentials into the new machine object, enabling certificate-based authentication or exploiting vulnerabilities like **CVE-2022-26923** to impersonate Domain Controllers.

3. Privilege Escalation & Persistence

- **Group Membership:** New computer objects are automatically added to the **Domain Computers** group. If this group has been granted local admin rights on specific servers or workstations, the attacker immediately inherits those permissions.
- **Persistence:** Rogue machine accounts provide a stable, long-term access point that is often harder for defenders to track than a standard user account.

Vulnerability Four: Machine Account Quota: Mitigation & Hardening Strategies

1. The Primary Fix: Set Quota to Zero

- **Action:** Use ADSI Edit (`adsiedit.msc`) to locate the `ms-DS-MachineAccountQuota` attribute on the domain object and change the value from **10** to **0**.
- **Result:** Immediately prevents non-privileged users from creating rogue computer objects, effectively blocking the entry point for RBCD and Shadow Credential attacks.

2. Delegated Administration

- **Controlled Access:** Instead of a domain-wide default, explicitly delegate the **"Create Computer Objects"** permission.
- **Scoped OUs:** Assign this right only to authorized personnel (e.g., Help Desk) and restrict it to specific Organizational Units (OUs) rather than the entire domain.

3. Monitoring & Auditing

- **Event Logging:** Monitor for **Windows Security Event ID 4741** (A computer account was created). Any instance of this event triggered by a non-admin account should be treated as a high-priority alert.
- **Identity Tracking:** Audit the `ms-DS-CreatorSID` attribute on existing machine accounts. This attribute reveals exactly which user created the computer object, allowing you to identify accounts spawned by unauthorized users.

Vulnerability Four: Machine Account Quota: The Logic Engine

```
def check_machine_account_quota(domains, flags):

    for domain in domains:

        properties = domain.get("Properties", {})

        domain_name = properties.get("name", "UnknownDomain")

        # Look for both naming formats
        quota = (
            properties.get("machineaccountquota")
            or properties.get("ms-DS-MachineAccountQuota")
            or 0
        )

        # Debug line (temporary)
        print("DEBUG:", domain_name, quota)

        if quota > 0:

            flags.append({
                "domain": domain_name,
                "vulnerability": "Machine Account Quota",
                "quota": quota,
                "description": f"{domain_name} allows users to create {quota} machine accounts."
            })
```

1. Identification Strategy

- **Target Data:** Unlike previous functions, this iterates through the `domains` array, as the quota is a domain-wide configuration rather than a per-user setting.
- **Property Extraction:** It targets the `Properties` block of the domain node to find the specific quota limit.

2. Robust Logic Handling

- **Flexible Naming:** The script is designed to handle different SharpHound output versions by checking for both the simplified key (`machineaccountquota`) and the formal LDAP attribute name (`ms-DS-MachineAccountQuota`).
- **Zero-Value Threshold:** It uses a logical `or 0` to ensure the script doesn't crash if the property is missing, then triggers the flag only if the value is greater than zero.

3. Reporting and Output

- **Domain-Level Flagging:** The output identifies the specific domain name and the exact quota limit allowed.
- **Contextual Alerting:** By including the actual number of allowed machines in the description, it helps administrators understand the scale of the risk (e.g., a quota of 10 vs. a quota of 100).

Vulnerability Five: Unpatched Domain Controllers (OS Version): Security Risks

1. Identification Strategy

- **Target Data:** The parser scans `computers.json`, but applies a critical filter: it only evaluates objects where the property `"isdc"` is set to `true`.
- **OS Attributes:** The logic utilizes two primary strings for verification:
  - `operatingsystem`: The "friendly" name (such as `"Windows Server 2012 R2"`).
  - `operatingsystemversion`: The specific build number (e.g., `"6.3 (9600)"`), which reveals the patch level or service pack status.

2. Detection Criteria

- **End-of-Life (EoL) Matching:** The script performs a substring match to identify legacy platforms that no longer receive security updates. Key targets include:
  - Windows Server 2000, 2003, and 2008/R2.
  - Windows Server 2012/R2 (critical as they recently exited extended support).
- **Build Number Analysis:** By checking the version string, the logic can identify if a modern OS (like Server 2016 or 2019) is running an early, unpatched build vulnerable to known exploits like **Zerologon** or **NoPac**.

3. Vulnerability Context

- **Legacy Exploits:** Systems running EoL software are permanently vulnerable to critical flaws (like **EternalBlue**) because vendor patches are no longer released.
- **Patch Gaps:** Even on supported versions, older build numbers indicate a failure in the patch management lifecycle, leaving the organization's "Identity Root" exposed to well-documented escalation paths.

Vulnerability Five: Unpatched Domain Controllers (OS Version): Security Risk & Impact Analysis

1. Critical Security Risks

- **The Identity Root at Risk:** Since Domain Controllers hold the entire identity infrastructure, a compromise here is a total domain takeover.
- **Weaponized Vulnerabilities:** End-of-Life (EOL) systems are permanent targets. Once a patch cycle ends, known flaws remain open forever, making them easy pickings for automated exploit kits.
- **Legacy Protocol Exposure:** Outdated OS versions often have insecure protocols like **NTLMv1** or **SMBv1** enabled by default, which were the primary vectors for devastating attacks like **WannaCry** and **EternalBlue**.
- **Broken Authentication Hardening:** Modern security updates (like Kerberos PAC signatures) can actually break communication between patched and unpatched DCs, leading to widespread authentication failures across the forest.

2. Operational & Performance Impacts

- **Security Software Blind Spots:** Modern EDR (Endpoint Detection and Response) and AV solutions often lack support for legacy kernels, leaving your most sensitive servers without advanced threat protection.
- **Instability and Inconsistency:** Running mixed OS versions or inconsistent Cumulative Update (CU) levels can lead to corrupted Kerberos tickets and replication errors between sites.
- **Performance Bottlenecks:** Older OS builds are not optimized for modern virtualization or hardware, causing slower authentication response times.

3. Legal & Compliance Dangers

- **Audit Failures:** Most regulatory frameworks (GDPR, HIPAA, PCI-DSS) explicitly forbid the use of unsupported software. An EOL Domain Controller is an automatic fail on a security audit.
- **Liability:** In the event of a breach, running known-vulnerable, unsupported systems can be classified as gross negligence, leading to significant financial penalties and legal liability.

Vulnerability Five: Unpatched Domain Controllers (OS Version): Mitigation & Hardening Strategies

1. Network Isolation & Control

VLAN Segmentation: Move outdated DCs to a dedicated, isolated VLAN to impede lateral movement.
Outbound Restrictions: Block all outbound internet access from DC subnets. If connectivity is essential, use a proxy with strict DNS filtering.
Firewall Hardening: Use host-based firewalls to block unauthenticated DCE/RPC traffic and limit communication to only necessary ports and authorized systems.

2. Protocol & Service Hardening

Kill Legacy Protocols: Disable **SMBv1** and **NTLMv1** immediately. Move toward restricting NTLM entirely in favor of Kerberos.
Service Minimization: Disable the **Print Spooler** service on all DCs to close off frequent privilege escalation paths.
Enforce Security Signing: Manually enforce **SMB signing** and **LDAP channel binding** to neutralize man-in-the-middle (MitM) relay attacks.

3. Administrative Security Model

Tiered Administration: Implement a **Tier 0** model for DCs. This ensures that high-privileged credentials never touch less secure workstations.
PAW Implementation: Require all DC management to be performed from **Privileged Access Workstations (PAW)**; dedicated, hardened endpoints used for no other purpose.
Identity Protection: Enable **Windows Credential Guard** or **LSA Protection** to safeguard credentials in memory from harvesting tools.

4. The Strategic Exit Plan

Avoid In-Place Upgrades: Don't upgrade the OS on an existing, potentially compromised DC.
Clean Rebuilds: Promote new servers running modern operating systems (e.g., Windows Server 2022) to the DC role.
Secure Decommissioning: Once roles are migrated, demote the legacy DC and perform a thorough **metadata cleanup** in Active Directory to remove all traces of the old system.

Vulnerability Five: Unpatched Domain Controllers (OS Version): The Logic Engine

1. Identification Strategy

- **Target Data:** The function processes the `computers` array from the SharpHound output, which contains metadata for all joined systems.
- **Property Focus:** It pulls the `operatingsystem` string from the `Properties` block to compare against a known list of vulnerable platforms.

2. Robust Version Matching

- **Substring Comparison:** Rather than looking for an exact match, the script iterates through an `outdated_versions` list. This is key because it catches various editions (such as"Standard," "Datacenter," or "R2") that share the same vulnerable kernel.
- **Broad Scope:** The list includes not just legacy Windows Servers, but also EoL client OS versions and common Linux distributions like Ubuntu 14/16 and CentOS 6/7 that might be hosting domain services.
- **Null Handling:** The script includes a safety check (`if not os_name: continue`) to ensure the parser doesn't crash if SharpHound was unable to resolve the OS for a specific object.

3. Reporting and Output

- **Domain Context:** The output links the vulnerable machine to its specific `domain_name`, which is vital for multi-domain forests.
- **Actionable Intelligence:** By including the exact OS string found in the description, it provides immediate evidence for IT teams to begin decommissioning or isolation workflows.

```
def check_outdated_os(computers, flags):

    outdated_versions = [
        # Vulnerable Systems
        "Windows XP",
        "Windows Vista",
        "Windows 7",
        "Windows 8",
        "Windows 8.1",

        # Vulnerable Servers
        "Windows Server 2003",
        "Windows Server 2008",
        "Windows Server 2008 R2",
        "Windows Server 2012",
        "Windows Server 2012 R2",
        "Windows Server 2016",

        # Vulnerable Non-Windows Systems
        "Ubuntu 14",
        "Ubuntu 16",
        "CentOS 6",
        "CentOS 7"
        ]

    for computer in computers:

        props = computer.get("Properties", {})

        computer_name = props.get("name", "Unknown")
        domain_name = props.get("domain", "UnknownDomain")

        os_name = props.get("operatingsystem")

        # Skip if OS is null
        if not os_name:
            continue

        print(f"DEBUG: {computer_name} - {os_name}")

        # Check partial match
        for outdated in outdated_versions:

            if outdated in os_name:

                flags.append({
                    "computer": computer_name,
                    "domain": domain_name,
                    "vulnerability": "Outdated OS",
                    "operatingsystem": os_name,
                    "description": f"{computer_name} in {domain_name} is running an outdated OS: {os_name}"
                })

                break
```